FIT1051 ASSIGNMENT ONE - TASK 3

Submission deadline: Before the start of Week 6 Applied Class, via Moodle, and to be demonstrated to the tutor during that class

Weight: 2% of your overall unit mark

Academic Integrity:

Please be reminded of the academic integrity standards that are expected of you at Monash. You should work alone and ask the unit staff for help if needed. Do not post your work in public forums or send your work to anyone. Do not copy/paste text or code from other sources and present it as your own — this includes use of generative AI tools. Breaching these academic integrity requirements can incur serious penalties. Please note, as per university requirements, similarity detection will be used across all student submissions. You may be asked for a short interview if there are any concerns regarding similarity or the use of Generative AI.

ASSIGNMENT ONE OVERVIEW:

Throughout this project, you will gradually develop a console-based Labyrinth Battle Simulator in Java. The project is divided into seven weekly tasks, each introducing new programming concepts such as:

- Classes and objects
- Encapsulation
- Lists and collections
- Control structures
- Modular programming
- Searching and managing objects
- Simulation logic

By the end of the project, you will have created a fully functional labyrinth adventure game where a player navigates through a maze, encounters enemies, and engages in simulated battles.

OVERVIEW OF THE LABYRINTH BATTLE SIMULATOR:

A Labyrinth Battle Simulator (LBS) is a console-based application that allows users to explore a labyrinth while encountering enemies and simulating battles. It represents a simple adventure-style system where a player navigates through different rooms, interacts with objects, and faces challenges along the way.

Creating a Player

The LBS allows the user to create a player character with basic attributes such as a name and combat abilities. This player represents the main character that will move through the labyrinth and participate in battles against enemies. The player's attributes determine how effective they are during combat.

Managing Enemies

The LBS keeps track of enemies that exist within the labyrinth. Each enemy has its own attributes such as a name, health level, and attack strength. These enemies are placed in different areas of the labyrinth and interact with the player when encountered.

Navigating the Labyrinth

The labyrinth acts as the environment of the game. It consists of multiple rooms or locations connected. The player can move between these rooms using menu options. Some rooms may contain enemies or other elements, while others may simply allow the player to progress further into the labyrinth.

Battle Simulation

The LBS simulates combat between the player and enemies. When the player encounters an enemy in the labyrinth, a battle sequence is triggered. During the battle, both the player and the enemy take turns attacking until one of them is defeated. Instead of graphical gameplay, the results of the battle are displayed through text output in the console.

Tracking Game Progress

The LBS can track important information about the game, such as player health, defeated enemies, or progress through the labyrinth. This allows the system to simulate a simple progression system where the player advances through different rooms and challenges.

Expanding the System

The LBS is designed to grow over time. Additional features such as new enemy types, inventory systems, treasures, or more complex labyrinth structures can be implemented as the system develops.

THIS WEEK'S TASK INSTRUCTIONS / REQUIREMENTS:

In the A1Task3.zip:

1. Create a `Room` class with the following fields:
    a. **id** – *which stores the room id (e.g. 1)*
    b. **exits** – *which is an array that stores exactly 4 directions in the order north, east, south and west. Each value should be: 0 if there is no exit in that direction, or the ID of the room that the exit leads to.*
    c. **monster** – *which stores the monster object that is present in the room*
    d. implement all standard components of the class, including constructors, getters, setters, and toString methods.
    e. Additionally, implement:
        i. **getExitAt(int):** returns the exit value at a specified index based on the fixed direction order. For example, getExitAt(0) should return the id of the room assigned to the north direction.
        ii. **setExit(int, int):** updates the exit value at a specified index. For example, setExit(1, 2) should set the east exit to the room with id 2.

FOR HD (*IN ADDITION TO ABOVE LISTED TASKS*)**:**

- **Input Validation**
  Modify the `**getStringInput(String)**` and `**getIntInput(String, int, int)**` methods in the `ConsoleIO` class so that they keep prompting the user until valid input is entered. This should also prevent the program from crashing when the user provides input of an incorrect type.
- Update the main menu so that it runs continuously using loops and only stops when the user selects **Quit / Exit**.
- Modular code that is easy to read and maintain.
- Follows good OOP and programming practices.

SUBMIT YOUR ASSIGNMENT ZIP WITH THE FOLLOWING NAME - <STUDENTID>_<NAME>_<APPLIEDSESSION_A1T3>.ZIP

Do's	Don'ts
• ADD CODE TO THE GIVEN PROJECT • UPLOAD YOUR SOLUTION TO MOODLE UNDER THE CORRECT A1 WEEKLY TASK SUBMISSION LINK. • ATTEND THE APPLIED SESSION ON YOUR TASK'S DUE DATE TO GET MARKED. • MUST USE THE PROJECT FILES PROVIDED	• MODIFY ANY EXISTING CODE IN THE TEMPLATE (EXCEPT COMMENTS). • SUBMIT THE WRONG PROJECT ZIP FILE. • MISS THE APPLIED SESSION — MARKING IS DONE IN CLASS ONLY. • USE YOUR OWN PROJECT FILES.